

COMICCRAFT

AI Comic Story Creator

01	Brainstorming & Ideation Phase
02	Requirement Analysis Phase
03	Project Design Phase
04	Project Planning Phase
05	Project Development Phase
06	Project Testing Phase
07	Project Documentation Phase
08	Project Demonstration Phase

Repository-ready documentation covering the complete ComicCraft lifecycle from initial idea through demonstration.

Technology scope: FastAPI, Jinja2, Gemini models for text generation, Hugging Face image generation, Pillow, FPDF, Python virtual environment, and Git/GitHub workflow.

Prepared for GitHub project documentation

Project Overview

ComicCraft is a web-based AI comic creation application. The user supplies a story prompt, main character name, setting, tone, and art style. The system turns those inputs into a structured five-panel comic story, generates illustrations, assembles the panels, and exports the result as a PDF.

The documented architecture separates the frontend, FastAPI backend, and AI integration services. This modular structure supports development, testing, replacement of AI providers, and future enhancements.

Project Flow

1	User Input	Prompt + character + setting + tone + art style
2	Story Outline	Structured five-panel outline
3	Narration	Panel narration and dialogue
4	Illustration	AI-generated comic panel images
5	Layout	Images + titles + story content
6	Export	Downloadable PDF comic

Document Basis

This phase document follows the eight phase titles requested for the GitHub repository and maps them to the milestones, activities, architecture, implementation, and deployment/testing material in the ComicCraft project documentation.

Purpose

Define the problem, creative concept, target experience, and expected end product for ComicCraft.

Activities

- Identify ComicCraft as an AI-assisted comic creation platform that converts a user idea into a structured comic story and illustrations.
- Explore the core user scenario: enter a story prompt, character name, setting, tone, and art style, then generate a complete comic.
- Consider iteration scenarios such as changing tone or art style and regenerating the comic.
- Define the final experience: panel-by-panel preview and downloadable PDF output.

ComicCraft Implementation

The project concept combines language generation for story creation with image generation for comic-style illustrations and a PDF export stage.

DELIVERABLES	COMPLETION EVIDENCE
• Project concept • User scenarios • Core feature list • Expected output definition	The project specification describes multiple generation scenarios and the end-to-end comic workflow.

Purpose

Translate the idea into functional requirements, technical prerequisites, inputs, outputs, and constraints.

Activities

- Define required user inputs: Story Prompt, Main Character Name, Setting, Story Tone, and Art Style.
- Define required outputs: five-panel outline, narration/dialogue, panel illustrations, comic layout, and PDF export.
- Identify technical prerequisites: Python, FastAPI, Uvicorn, Jinja2 templates, Gemini API, Hugging Face/Diffusers, Pillow, FPDF, and related packages.
- Define supporting capabilities such as form validation, JSON API access, image testing, and graceful error handling.

ComicCraft Implementation

ComicCraft uses a modular backend where the user request is validated, passed through AI generation services, assembled into a layout, and exported.

DELIVERABLES	COMPLETION EVIDENCE
• Functional requirements • Technical requirements • Input/output specification • Prerequisite list	The project specification explicitly lists required inputs, technical prerequisites, routes, and the generation pipeline.

Purpose

Design the system architecture, module responsibilities, data flow, interface structure, and integration points.

Activities

- Design a three-part architecture: frontend, FastAPI backend, and AI integration services.
- Assign responsibilities to the service modules: outline generation, story generation, image generation, layout building, and PDF export.
- Design Jinja2 templates for input, comic preview, export success, and error handling.
- Define the request flow from user form or JSON request through AI services and final export.

ComicCraft Implementation

The repository separates configuration, schemas, routes, AI services, templates, static assets, and tests so that each responsibility remains modular.

DELIVERABLES	COMPLETION EVIDENCE
• Architecture design • Module responsibilities • Data flow • UI/page structure	The source documentation describes the frontend/backend/AI architecture and the roles of the major Python modules and templates.

Purpose

Organize implementation work into an ordered sequence so that each stage can be developed and verified before the next.

Activities

- Plan the work as model selection and architecture, core development, routing, frontend development, deployment, and testing.
- Prepare the project folder structure for application code, services, templates, static assets, tests, and VS Code settings.
- Plan the development environment using a Python virtual environment and a requirements file.
- Plan validation through local server startup, API documentation, direct service tests, and end-to-end comic generation.

ComicCraft Implementation

The planning structure follows the documented milestones and was used to move ComicCraft from environment setup to a working local generation pipeline.

DELIVERABLES	COMPLETION EVIDENCE
• Development plan • Repository structure • Environment plan • Milestone sequence	The specification presents milestones for model selection, core functionality, routes, frontend, deployment, and verification.

Purpose

Implement the ComicCraft application and connect all major components into a working generation pipeline.

Activities

- Implement structured five-panel outline generation in `gemini_flash.py`.
- Implement story narration and dialogue generation in `gemini_pro.py`.
- Implement image generation in `image_generator.py` with configurable providers.
- Implement layout assembly and PDF export through `layout_builder.py` and `exporters.py`.
- Connect the workflow through FastAPI routes, form handling, JSON requests, templates, and static files.

ComicCraft Implementation

The application now runs locally with Python 3.12, Gemini text generation, configurable Hugging Face image generation, Jinja2 frontend pages, and PDF export. The working flow has been verified during local testing.

DELIVERABLES	COMPLETION EVIDENCE
• Backend modules • Frontend templates • AI integration • Image assets • PDF export • API routes	The documented project architecture and the current working local project demonstrate the implemented generation flow.

Key repository components: `app/main.py`, `app/config.py`, `app/routes.py`, `app/schemas.py`, `app/services/`, `templates/`, `static/`, `tests/`, `requirements.txt`, `.env.example`, and `README.md`.

Purpose

Verify that individual components and the complete ComicCraft workflow operate correctly.

Activities

- Verify the local FastAPI server starts successfully and the home page is reachable.
- Test Gemini API connectivity and confirm that the configured API key is loaded.
- Test outline generation independently and confirm that exactly five panels are returned.
- Test Hugging Face image generation independently before running the complete comic workflow.
- Run an end-to-end generation and verify that the comic reaches the preview/PDF stage.

ComicCraft Implementation

During project setup, the Gemini connectivity test returned a valid response, the outline service generated five panels, Hugging Face produced a test image, and an end-to-end five-page ComicCraft PDF was successfully generated.

DELIVERABLES	COMPLETION EVIDENCE
• Test cases • Component test results • Integration test results • Generated output for verification	The verified local run produced a five-page comic PDF containing five generated panels, narration, captions, and dialogue.

Verification sequence: server startup -> Gemini connectivity -> outline generation -> image generation -> full comic generation -> PDF inspection.

Purpose

Record the system, setup process, project workflow, code structure, requirements, testing procedure, and usage instructions.

Activities

- Document project description, scenarios, technical architecture, milestones, and development activities.
- Document setup commands, virtual environment creation, package installation, environment variables, and local server execution.
- Document the routes and templates so developers can understand how user input becomes a comic.
- Prepare repository-level documentation such as README.md, .env.example, requirements.txt, and test files.

ComicCraft Implementation

The project documentation can be kept in GitHub alongside the source code, with secrets excluded from version control and setup information kept reproducible.

DELIVERABLES	COMPLETION EVIDENCE
• README.md • Phase document PDF • .env.example • requirements.txt • Test documentation	The source project document already contains the technical architecture, workflow, deployment notes, and route/template documentation.

Purpose

Present the completed system clearly by demonstrating the user journey and the technical pipeline.

Activities

- Start the FastAPI application locally and open the ComicCraft homepage.
- Enter a sample story prompt with character, setting, tone, and art style.
- Demonstrate AI-generated five-panel story content and image generation.
- Show the comic preview and the final PDF export.
- Briefly explain the architecture, AI services, routes, testing, and future enhancement opportunities.

ComicCraft Implementation

A suitable demonstration can show the complete sequence from user input to generated comic, including the five-panel PDF output already verified during testing.

DELIVERABLES	COMPLETION EVIDENCE
• Live demonstration • Sample generated comic • PDF output • Project explanation	The documented project scenarios describe the intended demonstration flow, and the local project has already produced a working PDF output.

Demonstration sequence: show input form -> submit prompt -> explain AI pipeline -> display five-panel result -> open/export the PDF.

GitHub Upload Checklist

- Add the complete ComicCraft source tree and README.md.
- Keep .env.example in the repository, but do not commit real API keys or tokens.
- Keep generated temporary images/PDFs out of the repository unless they are intentionally included as demo assets.
- Include requirements.txt and clear Windows setup/run commands.
- Include the phase documentation PDF as a project documentation artifact.
- Add screenshots or a sample output PDF separately when needed for project presentation.

Suggested repository documentation layout

README.md	Project overview, features, setup, run, test, and usage instructions
docs/ComicCraft_Project_Phases.pdf	This phase-by-phase project lifecycle document
.env.example	Environment variable names without secrets
requirements.txt	Python dependencies
tests/	Automated tests for the application and layout builder

End of document.